SOMNGUARD

Documentación de Análisis del Software

Estado: En progreso **Fecha:** 2026-08-19

Contenido

- 1. Introducción
- 2. Descripción del sistema
- 3. Modelo de dominio
- 4. Casos de uso
- 5. Vistas estáticas
- 6. Vistas dinámicas
- 7. Análisis de requisitos por módulo
- 8. Trazabilidad

1. Introducción

1.1 Propósito del documento

El propósito de este documento es presentar el análisis del software SomnGuard: el modelo de dominio, los casos de uso, las vistas estáticas y las **vistas dinámicas** del sistema (secuencia, actividades y estados), como puente entre la especificación de requisitos (SRS) y el diseño detallado (informe de diseño de software).

1.2 Alcance

El análisis cubre los componentes de software de SomnGuard: plataforma cloud (backend, portal web, app móvil), dispositivo edge y sus interacciones. No cubre el diseño de hardware.

1.3 Documentos de referencia

Documento	Ubicación
SRS	01-srs/
Funcionalidades del sistema (épicas F-01..F-10)	../03-product-definition/product-backlog.md
Estructura módulo-entidad	../09-modules/module-catalog.md
Modelo de datos vigente	../06-data-architecture/02-modules-entities.md
Índice de diagramas	../08-uml/diagram-index.md
Informe de diseño de software	../05-architecture/software-design-report.md

2. Descripción del sistema

SomnGuard es un sistema de prevención de accidentes por fatiga o somnolencia al volante, compuesto por:

- **Dispositivo edge**: cámara y procesador embebido (Raspberry Pi) que monitorea al conductor, detecta estados de riesgo mediante visión por computadora (Python) y emite alertas sonoras inmediatas.
- **Plataforma cloud**: backend monolito modular (Java 21 + Spring Boot 4.1.1), base de datos PostgreSQL, portal web (React JS) y app móvil (React Native), que almacenan los eventos sincronizados, permiten su consulta/análisis y gestionan cuentas, dispositivos y catálogos.

Funcionalidades principales (detalle en product backlog):

#	Funcionalidad
F-01	Monitoreo continuo del conductor
F-02	Detección de estados de riesgo (somnolencia/fatiga)
F-03	Generación de alertas sonoras
F-04	Registro de eventos con evidencia
F-05	Transmisión de información a la plataforma (con soporte offline)
F-06	Análisis de datos y generación de reportes
F-07	Gestión de dispositivos (alta, asignación, configuración)
F-08	Notificaciones de eventos críticos
F-09	Gestión de cuentas, roles y permisos
F-10	Administración de catálogos (parameterization)

3. Modelo de dominio

El modelo de dominio agrupa **20 entidades** en 6 módulos, alineado con el modelo de datos vigente:

Módulo	Entidades
Security	user, role, module, feature, role_feature, user_role, password_reset_request, audit_login
Parameterization	event_category, severity, media_type, sound_pattern, event_type
Device Management	device, device_assignment, device_config
Telemetry Service	event, evidence, alert_log
Monitoring	notification
Analytics	vistas/reportes derivados (sin entidades transaccionales)

Diagrama de clases del dominio: cd-domain.mmd (exportación).

4. Casos de uso

Los casos de uso están modelados en 08-uml:

Diagrama	Descripción
`cu-general`	Actores y relación dispositivo ↔ plataforma
`cu-device`	Ciclo de vida del dispositivo, captura, alertas y sincronización
`cu-local-management`	Operaciones locales del dispositivo
`cu-account-device`	Registro, validación, login y asociación de dispositivos
`cu-plataforma`	Funcionalidades de la plataforma
`cu-analytics-visualization`	Módulo analítico: línea de tiempo, evidencia, métricas, resumen IA y reportes

5. Vistas estáticas

5.1 Estructura de módulos (backend)

```
com.somnguard
├── security
├── parameterization
├── device_management
├── telemetry_service
├── monitoring
├── analytics
└── platform        # Transversal, fuera de los módulos: errores, logging, observabilidad
```

Cada módulo se organiza con arquitectura hexagonal (puertos y adaptadores): `application/port/{in,out}`, `application/usecase`, `domain/{model,service}`, `adapter/in/{web,amqp}`, `adapter/out/{persistence,storage}` (ver documento de arquitectura y ADR-002).

5.2 Componentes por entorno

Entorno	Componente	Tecnología
Edge	Captura, análisis de visión, alertas, agente de sincronización	Raspberry Pi, Python
Cloud	API REST, base de datos, almacenamiento multimedia, notificaciones	Java 21, Spring Boot 4.1.1, PostgreSQL, Liquibase
Clientes	Portal web, app móvil	React JS, React Native

6. Vistas dinámicas

Las vistas dinámicas describen el comportamiento del sistema: qué flujos ejecuta, en qué orden y cómo cambia el estado de sus entidades.

6.1 Diagramas de secuencia

Fuentes y exportaciones en `../08-uml/diagrams/source/` y `../08-uml/diagrams/exports/`:

Flujo	Fuente	Exportación
Detección y alerta (edge)	sd-detection-alert.mmd	sd-detection-alert.png
Sincronización offline	sd-offline-sync.mmd	sd-offline-sync.png
Autenticación	sd-authentication.mmd	sd-authentication.png
Restablecimiento de contraseña	sd-password-reset.mmd	sd-password-reset.png
Alta y asignación de dispositivo	sd-device-registration.mmd	sd-device-registration.png
Consulta de eventos	sd-events-query.mmd	sd-events-query.png
Notificación de evento crítico	sd-critical-notification.mmd	sd-critical-notification.png
Generación de reporte	sd-report-generation.mmd	sd-report-generation.png

Secuencia: detección y alerta

```
sequenceDiagram
    participant CAM as Cámara (Edge)
    participant VIS as Análisis de Visión (Python)
    participant DET as Detector de Riesgo
    participant SPK as Altavoz
    participant CON as Conductor
    participant REG as Registro Local (Edge)

    CAM->>VIS: Frames capturados del conductor
    VIS->>VIS: Preprocesa y analiza señales (parpadeo, PERCLOS, cabeza)
    VIS-->>DET: Características extraídas
    DET->>DET: Evalúa patrón de fatiga / somnolencia

alt Riesgo detectado
    DET->>SPK: Activa alerta sonora (patrón configurado)
    DET->>REG: Registra evento + evidencia (imagen/video)
else Sin riesgo
    DET->>REG: Continúa monitoreo sin registrar
end

    SPK-->>CON: Alerta acústica inmediata
```

Secuencia: sincronización offline

```
sequenceDiagram
    participant REG as Registro Local (Edge)
    participant COL as Cola de Sincronización
    participant AGT as Agente de Sincronización
    participant API as API Backend (Telemetry Service)
    participant DB as PostgreSQL
    participant STO as Almacenamiento Multimedia

    REG->>COL: Encola evento + evidencia
    Note over AGT: Sin conectividad: el lote permanece en cola
    AGT->>API: POST /api/v1/telemetry/events (lote, API key)
    API->>API: Valida dispositivo e idempotencia (ID único)
    API->>DB: Persiste evento
    API->>STO: Almacena evidencia multimedia
    API-->>AGT: 201 Created (ids de eventos)
    AGT->>COL: Elimina eventos confirmados
    AGT->>API: GET /api/v1/devices/{id}/config
    API-->>AGT: Configuración básica vigente
```

6.2 Diagramas de actividades

Flujo	Fuente	Exportación
Detección de evento	ac-event-detection.mmd	ac-event-detection.png

Flujo	Fuente	Exportación
Sincronización offline	ac-offline-sync.mmd	ac-offline-sync.png
Generación de reporte	ac-report-generation.mmd	ac-report-generation.png
Registro de cuenta	ac-account-registration.mmd	ac-account-registration.png

Actividad: detección de evento

```
flowchart TD
    A[Capturar frames de la cámara] --> B[Preprocesar imagen]
    B --> C[Analizar señales: parpadeo, PERCLOS, movimientos de cabeza]
    C --> D{Riesgo detectado?}
    D -- No --> A
    D -- Sí --> E[Reproducir alerta sonora]
    E --> F[Registrar evento local]
    F --> G[Adjuntar evidencia multimedia]
    G --> H[Encolar para sincronización]
    H --> A
```

Actividad: sincronización offline

```
flowchart TD
    A[Cola local con eventos pendientes] --> B{¿Hay conectividad?}
    B -- No --> A
    B -- Sí --> C[Enviar lote a la API con API key]
    C --> D{¿API confirma 201?}
    D -- No --> A
    D -- Sí --> E[Eliminar eventos confirmados de la cola]
    E --> F[Descargar configuración básica del dispositivo]
    F --> G[Aplicar configuración local]
```

6.3 Diagramas de estados

Entidad	Fuente	Exportación
Dispositivo	es-device.mmd	es-device.png
Evento	es-event.mmd	es-event.png
Cuenta	es-account.mmd	es-account.png

Estados del dispositivo

```
stateDiagram-v2
    [*] --> Registrado: Alta manual o self-register (UNREGISTERED = previo, sin fila en BD)
    Registrado --> Asignado: Asignación a una cuenta (assign admin o claim usuario)
    Asignado --> Activo: Primer heartbeat válido (saludo con API key)
    Activo --> Offline: Sin heartbeat 5min
    Offline --> Activo: Heartbeat válido (se restablece la conectividad)
    Activo --> Registrado: Desasociación (unassign)
    Asignado --> Registrado: Desasociación (unassign)
    Activo --> Suspendido: Administrador suspende
    Suspendido --> Activo: Administrador reactiva
    Suspendido --> Retirado: Fin del ciclo de vida
    Registrado --> Retirado: Alta cancelada
    Retirado --> [*]
```

Estados del evento

```
stateDiagram-v2
    [*] --> Detectado: Análisis de visión identifica un riesgo
    Detectado --> Registrado: Alerta emitida y evento guardado en cola local
```

```
    Registrado --> Sincronizado: Lote confirmado por la API
    Registrado --> Registrado: Sin conectividad (permanece en cola)
    Sincronizado --> Analizado: Consultado en la plataforma o incluido en reporte
    Sincronizado --> Archivado: Política de retención cumplida
    Analizado --> Archivado
    Archivado --> [*]
```

Estados de la cuenta

```
stateDiagram-v2
    [*] --> Pendiente: Registro de cuenta
    Pendiente --> Activa: Validación de correo confirmada
    Pendiente --> Eliminada: Validación caducada
    Activa --> Suspendida: Administrador suspende
    Suspendida --> Activa: Administrador reactiva
    Activa --> Eliminada: Baja voluntaria o administrativa
    Suspendida --> Eliminada
    Eliminada --> [*]
```

---

7. Análisis de requisitos por módulo

Módulo	Funcionalidades relacionadas	Casos de uso	Diseño (informe)
Security	F-09	cu-account-device	Autenticación JWT, roles y permisos
Parameterization	F-10	cu-plataforma	Catálogos configurables
Device Management	F-07	cu-device, cu-plataforma	Alta, asignación y configuración
Telemetry Service	F-01, F-02, F-03, F-04, F-05	cu-device, cu-local-management	Eventos, evidencia, alertas, sincronización
Monitoring	F-08	cu-plataforma	Notificaciones de eventos críticos
Analítico	F-06	cu-analytics-visualization	Línea de tiempo, métricas, resumen IA, reportes

---

8. Trazabilidad

Funcionalidad	Caso de uso	Vistas dinámicas	Módulo backend	Pruebas (estrategia)
F-01 Monitoreo continuo	cu-device	sd-detection-alert, ac-event-detection	Telemetry Service + Edge	Unitarias del detector; integración del agente
F-02 Detección de riesgo	cu-device	sd-detection-alert, es-event	Edge + Telemetry Service	Validación con datos de campo
F-03 Alertas sonoras	cu-device	sd-detection-alert	Edge	Pruebas en prototipo
F-04 Registro de eventos	cu-local-management	sd-offline-sync, es-event	Telemetry Service	Unitarias de repositorio; integración
F-05 Transmisión a plataforma	cu-general	sd-offline-sync, ac-offline-sync	Telemetry Service	Integración con API simulada
F-06 Análisis y reportes	cu-analytics-visualization	sd-report-generation, ac-report-generation	Analítico	Pruebas de API y E2E del portal

Funcionalidad	Caso de uso	Vistas dinámicas	Módulo backend	Pruebas (estrategia)
F-07 Gestión de dispositivos	cu-device, cu-plataforma	sd-device-registration, es-device	Device Management	Unitarias e integración
F-08 Notificaciones	cu-plataforma	sd-critical-notification	Monitoring	Pruebas de integración del push
F-09 Cuentas y roles	cu-account-device	sd-authentication, sd-password-reset, es-account	Security	Unitarias de seguridad; integración
F-10 Catálogos	cu-plataforma	—	Parameterization	CRUD por API

Documento de análisis alineado con el SRS, el modelo de datos vigente (20 entidades) y los diagramas UML publicados en `08-uml`.